

Proyecto Integrador — Gestor Automatizado de Servicios con Notificación por Telegram

Ingeniería en Ciberseguridad e Infraestructura de Cómputo *Programación para Administración de Servicios*

Sistema automatizado mediante scripts en Bash para la gestión integral de servicios en sistemas GNU/Linux. Cubre gestión de usuarios, respaldos automáticos, monitoreo de recursos (CPU/disco), supervisión de servicios con `systemd`, monitoreo de red (ping y puertos) y ejecución remota de scripts vía SSH/SCP. Todas las operaciones se registran en una bitácora centralizada y envían notificaciones en tiempo real a un bot de Telegram.

Arquitectura y Entorno de Pruebas

El proyecto utiliza un laboratorio multi-contenedor basado en Docker con red interna estática para simular un entorno de administración real.

Infraestructura

- Dockerfile:** Basado en `debian:12-slim` con `systemd` como proceso init (`/sbin/init`). Instala las dependencias del proyecto (`systemd`, `cron`, `curl`, `bc`, `iputils-ping`, `netcat-openbsd`, `nmap`, `openssh-client/server`, `procps`, `sudo`, `nano`, `iproute2`, `tar`), limpia unidades de `systemd` innecesarias para contenedores, habilita los servicios `ssh` y `cron`, y crea el usuario de pruebas `supervisor` (contraseña: `password`) con `sudo` sin contraseña. Incluye directorios de prueba y el archivo de bitácora preconfigurado.
- docker-compose.yml:** Define 4 contenedores interconectados en la red `redProyecto` (`172.20.0.0/16`):

Contenedor	Nombre	IP	Rol
debian-lab	proyecto_admon_cliente	172.20.0.2	Cliente principal (monta <code>/workspace</code>)
debian-lab2	proyecto_admon_servidor	172.20.0.5	Servidor remoto 1
debian-lab3	proyecto_admon_servidor1	172.20.0.6	Servidor remoto 2
debian-lab4	proyecto_admon_servidor2	172.20.0.7	Servidor remoto 3

Todos los contenedores corren en modo **privilegiado** con acceso a `cgroups` para soportar `systemd`.

Cómo levantar el laboratorio

1. Construir e iniciar todos los contenedores:

```
docker compose up -d --build
```

2. Entrar al contenedor cliente:

```
docker compose exec debian-lab bash
```

3. Entrar a un servidor remoto (ejemplo):

```
docker compose exec debian-lab2 bash
```

Configuración Global (**config.txt**)

Archivo de configuración central que unifica las variables utilizadas por todos los scripts. Está organizado en las siguientes secciones:

Sección	Variables
Telegram	TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
Logs	LOG_FILE (por defecto /var/log/gestion_automatizada.log)
RespalDOS	BACKUP_SOURCE_DIRS, BACKUP_DEST_DIR, BACKUP_PREFIX
Monitoreo	CPU_THRESHOLD, DISK_THRESHOLD, DISK_PATHS
Servicios	SERVICES (lista de servicios systemd a monitorear)
Remoto	REMOTE_HOSTS_FILE, REMOTE_SCRIPT_LOCAL, REMOTE_USER, REMOTE_SSH_PORT, REMOTE_SSH_KEY, REMOTE_TARGET_DIR, REMOTE_REPORT_DIR, REMOTE_CONNECT_TIMEOUT
Red	NETWORK_HOSTS (formato IP:puerto1,puerto2), CRITICAL_PORTS
Cron	CRON_SCHEDULE

Scripts

Todos los scripts se encuentran en el directorio **scripts/**. Cada uno es funcional de forma independiente, lee su configuración desde **config.txt**, registra acciones en la bitácora central y notifica eventos relevantes por Telegram.

1. Gestión de Usuarios — **scripts/usuarios.sh**

Administración interactiva de usuarios del sistema. Requiere privilegios de superusuario (**sudo**).

- **Características:**

- Validación estricta de nombres de usuario mediante expresión regular (`^[a-z_][a-z0-9_-]{0,31}$`).
 - Comprobación de existencia previa antes de crear/eliminar/modificar.
 - Confirmación interactiva antes de eliminar cuentas.
 - Registro en bitácora y notificación por Telegram de cada operación.
 - **Menú interactivo** (`sudo ./usuarios.sh`):
 1. **Crear usuario** — con directorio home, GECOS y contraseña inicial.
 2. **Eliminar usuario** — eliminación segura con `userdel -r` tras confirmación.
 3. **Modificar usuario** — submenú con opciones: cambiar shell, GECOS, contraseña, añadir/quitar grupos.
 4. **Listar usuarios** — muestra los usuarios del sistema.
 5. **Salir**.
-

2. RespalDOS Automáticos — `scripts/respaldo.sh`

Automatiza la compresión de directorios con `tar`, verifica los resultados y programa la ejecución periódica con `cron`.

- **Características:**
 - Verificación de existencia de los directorios de origen.
 - Validación del archivo comprimido generado (existencia y tamaño > 0).
 - Bitácora con marcas de tiempo ISO-8601.
 - Notificación por Telegram con ruta, tamaño y fecha del respaldo.
 - Instalación automática en el `crontab` del usuario actual (sin requerir `root`).
 - **Menú interactivo** (`./respaldo.sh`):
 1. Ejecutar respaldo ahora
 2. Programar respaldo con cron
 3. Mostrar configuración
 4. Salir
 - **Argumentos CLI:**
 - `--backup-now` — Ejecuta respaldo inmediatamente.
 - `--install-cron` — Instala la tarea en el crontab.
 - `--show-config` — Muestra la configuración vigente.
-

3. Monitoreo de Recursos — `scripts/monitoreo.sh`

Monitorea el uso de CPU y disco, registra cada lectura y envía alertas por Telegram al superar los umbrales.

- **Características:**
 - Lectura de CPU mediante muestreo de `/proc/stat` (cálculo de porcentaje real).
 - Lectura de disco con `df` para múltiples particiones.
 - Umbrales configurables vía `config.txt` o argumentos CLI (por defecto 70%).
 - Modo ejecución única (`--once`, por defecto) o modo continuo (`--interval`).
 - Manejo de señales (`SIGINT`, `SIGTERM`) para salida limpia.
- **Argumentos CLI:**
 - `--cpu N` — Umbral de CPU (%).

- `--disk N` — Umbral de disco (%).
- `--paths P1,P2` — Particiones a monitorear (separadas por comas).
- `--interval S` — Intervalo en segundos (activa modo continuo).
- `--once` — Ejecutar una sola lectura y salir.

- **Ejemplo:**

```
./monitoreo.sh --cpu 80 --disk 90 --paths "/",/home"  
./monitoreo.sh --interval 30
```

4. Supervisión de Servicios — `scripts/servicios.sh`

Revisa el estado de servicios `systemd`, reinicia automáticamente los inactivos y notifica los resultados.

- **Características:**

- Lee la lista de servicios desde `config.txt` (variable `SERVICES`) mediante parsing directo con `grep/sed`.
- Verifica existencia de cada servicio en `systemd` antes de consultar su estado.
- Reinicio automático con `systemctl restart` (usa `sudo` si no es root).
- Verificación post-reinicio y reporte de éxito o fallo.
- Registro completo en bitácora y notificación por Telegram.

- **Modo de uso:**

```
sudo ./servicios.sh      # Ejecutar revisión  
./servicios.sh -h        # Mostrar ayuda
```

5. Ejecución Remota — `scripts/remoto.sh`

Copia un script local a hosts remotos por SCP, lo ejecuta por SSH y genera reportes individuales por host.

- **Características:**

- Lee hosts desde un archivo externo (uno por línea, soporta comentarios `#`).
- Copia el script vía `scp` y lo ejecuta remotamente vía `ssh` en modo batch (`BatchMode=yes`).
- Limpieza automática del script remoto tras la ejecución.
- Soporte para autenticación por llave SSH o contraseña.
- Validación de formato de host, existencia de archivos y parámetros numéricos.
- Genera un reporte individual por cada host con: estado, código de salida, timestamp y salida remota.
- Genera un archivo resumen con totales de éxitos y fallos.
- Timeout de conexión configurable.

- **Argumentos CLI:**

- `-f, --hosts FILE` — Archivo de hosts/IPs.
- `-s, --script FILE` — Script local a copiar y ejecutar.
- `-u, --user USER` — Usuario SSH remoto.

- `-p, --port PORT` — Puerto SSH (por defecto 22).
- `-i, --identity FILE` — Llave privada SSH.
- `-d, --remote-dir DIR` — Directorio remoto temporal (por defecto `/tmp`).
- `-o, --output-dir DIR` — Directorio base de reportes.
- `-t, --timeout SEG` — Timeout de conexión en segundos.

- **Ejemplo:**

```
./remoto.sh -f /workspace/hosts.txt -s /workspace/scripts/holaMundo.sh
./remoto.sh -f hosts.txt -s script.sh -u supervisor -i ~/.ssh/id_ed25519
```

6. Monitoreo de Red — `scripts/red.sh`

Verifica la conectividad de hosts mediante `ping` y verifica puertos con `nc` (netcat). Clasifica cada host y alerta sobre puertos críticos caídos.

- **Características:**

- Lee hosts desde `config.txt` (variable `NETWORK_HOSTS`, formato `IP:puerto1,puerto2`) o desde un archivo externo.
- Verificación de conectividad con `ping`.
- Verificación de puertos abiertos con `nc -z`.
- Clasificación de cada host: **ACCESIBLE, PARCIALMENTE ACCESIBLE, SIN PUERTOS DISPONIBLES** o **SIN RESPUESTA**.
- Alertas por Telegram cuando un host no responde o un puerto crítico (`CRITICAL_PORTS`) está cerrado.
- Registro detallado en bitácora con puertos abiertos vs totales.

- **Menú interactivo (`./red.sh`):**

1. Ejecutar monitoreo
2. Mostrar configuración
3. Salir

- **Argumentos CLI:**

- `--check` — Ejecutar monitoreo directamente.
- `--show-config` — Mostrar configuración de red.

- **Ejemplo:**

```
./red.sh --check
```

Notas

Permisos del archivo de log

El Dockerfile crea automáticamente el archivo de bitácora y asigna la propiedad al usuario `supervisor`. Si necesitas recrearlo manualmente:

```
sudo touch /var/log/gestion_automatizada.log
sudo chown supervisor:supervisor /var/log/gestion_automatizada.log
```

Configuración SSH para ejecución remota

Para usar `remoto.sh` entre los contenedores del laboratorio, es necesario generar las llaves SSH en el contenedor cliente y copiarlas a los servidores:

```
# En el contenedor cliente (172.20.0.2)
ssh-keygen -t ed25519 -f /home/supervisor/.ssh/id_ed25519 -N ""
ssh-copy-id -i /home/supervisor/.ssh/id_ed25519 supervisor@172.20.0.5
ssh-copy-id -i /home/supervisor/.ssh/id_ed25519 supervisor@172.20.0.6
ssh-copy-id -i /home/supervisor/.ssh/id_ed25519 supervisor@172.20.0.7
```